

C++ Language Tutorial

★ Information of C++

- The C++ language was created by 'Bjarne Stroustrup' in 1979.
- The name of C++ is 'Classes with C'.
- He renamed C++ in 1983.
- C++ is Object Oriented Language.
- C++ - Fast program, more control over system resources, memory management.
- High performance.

Basic Structure [Hello World] [tw]

```
#include <iostream> - - - Header file
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    cout << "Hello World" << endl;
```

```
    return 0;
```

```
}
```

★ High level language

- Programmer friendly
- less memory efficient.
- Debugging is easy.
- It need compiler for translation
- C++, Python, Java

★ Low level language

- Machine friendly
- More memory efficient
- Debugging is complex
- It need assembler for translation
- Machine language

Variables

- Variable are containers for storing data values.

Syntax :

type variable name = value ;

eg.

```
int myNum = 15 ;  
cout << myNum ;
```

- ① Int - No. without decimals - eg. 123, 23, -1
- ② double - No. with decimals - eg. 1.5, 2.9901.
- ③ char - Single char. - eg. 'a' or 'B'
Store in single quote.
- ④ String - text. - eg. "Hello World".
Store in double quote.
- ⑤ bool - True or False - 0 & 1.

Comments. [explanatⁿ of code]

- Single line - `//`
- Multi line - `/* ... */`

Variable Scope.

- Scope of a variable is the region in code where the existence of variable is valid.

- ① Local Variable - It define within or inside the function or block - `{ }` - block
- ② Global Variable - It define outside fn & block.

```
eg. #include <iostream>  
using namespace std;  
int x = 0; // Global x.  
int main() {  
    int x = 10;  
    cout << x "value of global x";  
    return 0;  
}
```

cout << "Value of local x";

C++ Data Types

- Data types defines the type of data a variable can hold.

Built in

↓
int
float
double
bool
char

User Defined

↓
struct
union
Enum

Derived

↓
Array
Function
Pointer

Basic Input / Output in C++

- Streams : In C++ seq. of bytes in ~~it~~ corres-ponding to I/p & o/p.
- Input Streams : Directⁿ of flow of bytes takes place from I/p device to main memory.
- Output Streams : Directⁿ of flow of bytes takes place ~~from~~ from main memory to o/p device.

eg. #include <iostream>
using namespace std;

int main()

{

int a, b;

c = a + b;

cout << "Enter the value of a : " << endl;

cin >> a;

cout << "Enter the value of b : " << endl;

cin >> b;

cout << "Enter the sum : " << endl;

cin >> c;

return 0;

}

Note :

<< - insertion operator

>> - extraction operator

Data Type	SIZE (in Bytes)	Range
Short int	2	-32,768 to 32,767
unsigned int	4	0 to 4,294,967,295
int	4	-2,147,483,648 to 2,147,483,647
long int	4	
Signed char	1	-128 to 127
unsigned char	1	0 to 255
float	4	
double	8	
long double	12	

Header files

① User defined - #include "this.h"

[This will produce error if is not present in folder / file]

② System - #include <iostream>;

[comes with compiler]

Operators .

(tut 4)

```
#include <iostream>
```

```
using namespace std;
```

```
int main ()
```

```
{
    int a=4, b=5;
```

```
// Arithmetic Operators
```

```
    cout << a+b;
```

```
    cout << a-b;
```

```
    cout << a * b;
```

```
    cout << a / b;
```

```

cout << a % b;
cout << a++ ;    [first print then increment]
cout << a ;
cout << ++a ;    [first increment then print]
cout << --a ;

```

// Assignment Operator

```

int a = 3 ; b = 9 ;
char d = 'd' ;

```

// Comparison Operator

```

cout << (a == b) << endl ;
cout << (a != b) ;
cout << (a > b) ;
cout << (a < b) ;
cout << (a >= b) ;
cout << (a <= b) ;

```

// logical Operator

```

cout << ((a == b) && (a < b)) ;
cout << ((a == b) || (a > b)) ;
cout << (! (a == b)) ;

```

return 0 ;

Addition Program [Tut 2]

```

#include <iostream>
using namespace std ;

```

int main ()

```

{
    int a, b, c ;

```

```

    cout << "The value of a : " << endl ;    cin >> a ;

```

```

    cout << "The value of b : " << endl ;    cin >> b ;

```

```

c = a + b ;
cout << "The sum is : " << c ;
return 0 ;
}

```


Scope Resolution Operator. [tut 5]

```
#include <iostream>
using namespace std;
```

```
int c = 45; // Global V.
```

```
int main()
```

```
{
```

```
    int a, b, c;
```

```
    cout << "a:" << endl;
```

```
    cin >> a;
```

```
    cout << "b:" << endl;
```

```
    cin >> b;
```

```
    c = a + b;
```

```
    cout << c << endl;
```

```
    cout << "The global c is:" << c << endl;
```

```
    return 0;
```

```
}
```

- To access a global variable when there is a local variable with same name.

float, double & long double Literals [tut 5]

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    float f = 34.5 F;
```

```
    long double e = 34.5 L;
```

```
    cout << "The size of 34.5 is" << sizeof(34.5) << endl;
```

```
    cout << "The size of 34.5F is" << sizeof(34.5 F) << endl;
```

```
    cout << "The size of 34.5F is" << sizeof(34.5 F) << endl;
```

```
    cout << "The size of 34.5L is" << sizeof(34.5 L) << endl;
```

```
    return 0;
```

```
}
```

Reference Variable - one refers to address of another variable [tut 7]

```
#include <iostream>
using namespace std;

int main()
{
    float x = 455;
    float &y = x;
    cout << x << endl;
    cout << y << endl;
    return 0;
}
```

Typcasting - conversion of one data type to another in program [tut 8]

```
#include <iostream>
using namespace std;

int main()
{
    int a = 45;
    float b = 45.46;

    cout << "The value of a is: " << (float) a;
    cout << "The value of a is: " << float(a);
    cout << "The value of b is: " << int(a);
    return 0;
}
```

Constants.

- It is used to constants the value remaining entire program.
- It is not change & get error if you one changed it.

```
#include <iostream>
using namespace std;
```

```
int main()
{
    const int a = 3;
    cout << a;
    a = 4;
}
```

```
cout << a;
return 0;
```

output : error occurs.

Manipulators

- Manipulators are helping fn that can modify the I/O stream.
- All Manipulators are available on Internet.
- setw & endl, etc.

```
#include <iostream>
using namespace std;
int main()
```

```
{ int a=3, b=32, c=1234;
```

```
cout << setw(4) << a << endl;
```

```
cout << setw(4) << b << endl;
```

```
cout << setw(4) << c << endl;
```

```
return 0;
```

```
}
```

Operator Precedence

- Operator Precedence determines the grouping of terms in an expression.

```
#include <iostream>
using namespace std;
```

```
int main()
```

```
{ int a=3, b=4;
```

```
int c = (a*5) + b - 45 + 87;
```

```
cout << c;
```

```
return 0;
```

```
}
```


Pointers [Tutorial]

- Pointers holds the address of other data types.
- It is used to store the address.

'&' : Address of operator

'*' : Dereference of operator

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int a = 3;
```

```
    int * b = &a;
```

```
    cout << "The address of a is : " << &a << endl;
```

```
    cout << "The address of a is : " << b << endl;
```

```
    cout << "The value of address of b is : " << *b  
    << endl;
```

```
    return 0;
```

Arrays [Tutorial]

- It is collection of items of similar type stored in contiguous memory location.

0	1	2	3	← Index
5	4	6	9	

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{  
    int marks[4] = {89, 89, 89, 88};
```

```
    cout << marks[0] << endl;
```

```
    cout << marks[1] << endl;
```

```
    cout << marks[2] << endl;
```

```
    return 0;
```

Using loop (Arrays) :

```
#include <iostream>
using namespace std;

int main()
{
    int marks[3] = {99, 98, 96};
    for (int i = 0; i <= 3; i++)
    {
        cout << "The value of marks " << i << " is "
              << marks[i] << endl;
    }
    return 0;
}
```

Pointers & Arrays .

```
#include <iostream>
using namespace std;

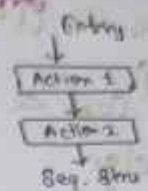
int main()
{
    int marks[3] = {99, 98, 96};
    // int *p = marks;
    // cout << "The value of *p is " << *p << endl;
    // cout << "The value of *(p+1) is " << *(p+1) << endl;
    // cout << "The value of *(p+2) is " << *(p+2) << endl;
    OR
    int *p = marks;
    cout << *(p+0);
    cout << *(p+1);
    return 0;
}
```

0	1	2
99	98	96

C++ Control Structures

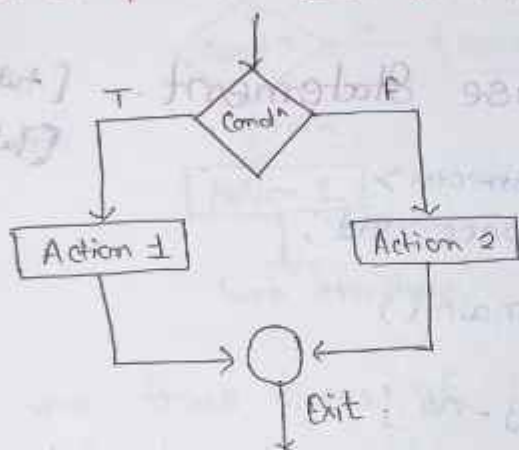
#1. Sequence Structure

- It is simple linear structure / instructions.
- No decision making & no loop.
- It executes linearly line by line.



#2.

- Sometimes in our program, we will need to write certain condition checks that this part of code should only execute if a particular condition meets or another part of code should run.



#2.1 IF-Else Statement (ladder). [Aut 11]

```
#include <iostream>
using namespace std;
```

```
int main()
```

```
{
```

```
    int age;
```

```
    cout << "Tell me your age : " << endl;
```

```
    cin >> age;
```

```
    if (age < 18)
```

```
    {
        cout << "You are Virat fan" << endl;
```

```
    }
```

```

else if ((age > 18) && (age < 45))
{
    cout << "Your Hitman 45 fan's" << endl;
}
else if ((age > 45) && (age < 63))
{
    cout << "Your Boom fan's" << endl;
}
else
{
    cout << "Your Dhoni fan's" << endl;
}
return 0;
}

```

2.2 Switch - Case Statement [tut 12] [tut 15]

```

#include <iostream>
using namespace std;

int main()
{
    int jersey-no;
    cout << "Enter Your Jersey-no." << endl;
    cin >> jersey-no;

    switch (jersey-no)
    {
        case 45 :
            cout << "Your Hitman 45 fan's";
            break;

        case 18 :
            cout << "Your Virat 18 fan's";
            break;

        case 7 :
            cout << "Your Mahi 7 fan's";
            break;
    }
}

```

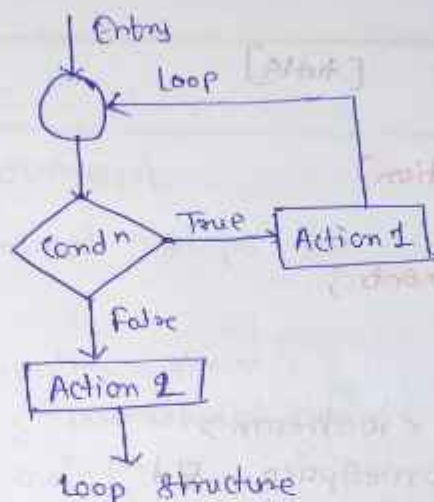

default :

```
cout << " Your HP83 fan's " << endl;  
break ;
```

```
}  
return 0 ;
```

```
}
```

3. Loop Structures.



• There are three types of loops in C++.

1. For loop.

2. While loop.

3. do-while loop.

1. For loop. (tut 13)

Syntax :

```
for (initialization ; condition ; updation)
```

```
{  
    loop body ;
```

```
}
```

```
#include <iostream>
using namespace std;

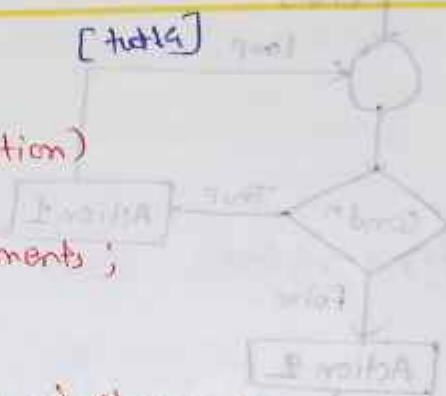
int main()
{
    for (int i=0; i<40; i++)
    {
        cout << i << endl;
    }
    return 0;
}
```

② While loop

Syntax :

While (condition)

```
{
    C++ statements;
}
```



```
#include <iostream>
using namespace std;
```

```
int main()
{
    int i=1;
    while (i<=40)
    {
        cout << i << endl;
        i++;
    }
    return 0;
}
```

② do-while loop

Syntax :

```
do
{
    C++ stat.;
} while (condition)
```



```
#include <iostream>
using namespace std;
int main()
{
    int i = 1;
    do
    {
        cout << i << endl;
        i++;
    } while (i <= 4);
    return 0;
}
```

Break Statement

- Break statement stops the entire process of loop.

```
#include <iostream>
using namespace std;
int main()
{
    for (int i = 0; i <= 4; i++)
    {
        cout << i << endl;
        i++;
        if (i == 2)
        {
            break;
        }
    }
    return 0;
}
```

Continue Statement

- Continue statement only stops the current iteration of loop.

Structures in C++ (Tut 17)

- It is used to combine diffⁿ types of data types.
- keyword = struct.

Syntax:

```
struct
```

```
{ // Declaration
```

```
}
```

```
#include <iostream>  
using namespace std;
```

```
struct Cric-player
```

```
{  
    int jersey-no;  
    int chor fowchor;  
    float runs;  
    String name;  
};
```

```
int main()
```

```
{  
    struct Cric-player R045;  
    R045.jersey-no = 45;  
    R045.runs = 1000000;  
    R045.fowchor = 'R';
```

```
    cout << R045.jersey-no << endl;
```

```
    cout << R045.runs << endl;
```

```
    cout << R045.fowchor << endl;
```

```
    return 0;
```

```
}
```


data types.

```
// using typedef
typedef struct employee
{
    int eid;
    char fvarchar;
    float salary;
} ep;

int main()
{
    ep harry;
    :
    :
}
```

#unions in C++ (tut 18)

- The union can contain diffⁿ types of data types.
- Each time a new variable is initialized.
- Keyword = union.

```
#include <iostream>
```

```
using namespace std;
```

```
union money
```

```
{
    int rice;
```

```
    char cor;
```

```
    float pounds;
}
```

```
}
```

```
int main()
```

```
{
```

```
    union money m1;
```

```
    m1.rice = 84;
```

```
    m1.cor = 'c';
```

```
    cout << m1.cor; return 0;
```

```
}
```

Enum in C++ (tut 19)

- It helps to assign constants to set of names to make program easier to read, maintain & understand.
- keyword 'enum'.

```
#include <iostream>
using namespace std;

int main()
{
    enum Meal { breakfast, lunch, dinner };
    cout << breakfast;
    cout << lunch;
    cout << dinner;
    return 0;
}
```

Functions in C++ (Revision) (tut 20)

- A fn is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a fn.
- main() - function.

Syntax :

```
void myfun ()
{
}
```

```
#include <iostream>
using namespace std;

int sum (int a, int b)
{
    int c = a + b;
    return c;
}
```



```

int main ()
{
    int num1, num2;
    cout << "Enter the num1: " << endl;
    cin >> num1;
    cout << "Enter the num2: " << endl;
    cin >> num2;
    cout << "Enter the Sum: " << sum (num1, num2);
    return 0;
}

```

Function Prototype in C++ (tut 21)

- It is basically a declared of fⁿ that tells the program about type of value which is to be returned by fⁿ.

return_type function_name (datatype-1, datatype-2, ...)

```
#include <iostream>
```

```
using namespace std;
```

```
int sum (int a, int b);
```

```
int main ()
```

```
{
```

```
    int num1, num2;
```

```
    cout << "Enter 1st no.:" << endl;
```

```
    cin >> num1;
```

```
    cout << "Enter 2nd no.:" << endl;
```

```
    cin >> num2;
```

// num1 & num2 are - actual parameters

```
    cout << "Sum is: " << sum (num1, num2);
```

```
    return 0;
```

```
}
```

// formal
parameters
a & b

```
int sum (int a, int b) {
```

```
    int c = a + b;
```

```
    return c;
```

```
}
```

Call by Value (see love babbar fn video) (Rev)

- A copy of value is passed to the function.
- Changes in inside fn is not reflected on outside fn.
- Actual & formal argument will be created in diff memory locatⁿ.

Call by reference

- An address of value is passed to fn.
- changes made inside the fn is reflected outside fn.
- Actual & formal argu. will be created in same memory locatⁿ.

#include <iostream> (tut 22)

using namespace std;

void SwapPointer (int *a, int *b)

```
{ int temp = *a; // temp a b
  *a = *b;       // 4 4 5
  *b = temp;     // 4 5 4
}
```

int main ()

{

int x=4, y=5

cout << "x: " << x << "y: " << y << endl;

SwapPointer (&x, &y);

cout << "x: " << x << "y: " << y << endl;

return 0;

}

Function Overloading (tut 23)

- Two or more fⁿ can have same name but diffn parameters.

```
#include <iostream>
using namespace std;

void add (int a, int b)
{
    cout << "Sum = " << (a+b);
}

void add (double a, double b)
{
    cout << endl << "Sum = " << endl << (a+b) << endl;
}

int main()
{
    add (10, 2);
    add (5.3, 6.2);
    return 0;
}
```

Inline functions. (Revision) (tut 24)

- C++ provides inline fⁿ to reduce the function call overhead.
- An inline fⁿ is fⁿ that is expanded in line when it is called.

Syntax

```
inline return-type function-name(parameters)
{
    // body
}
```

Compiler

- May not perform inlining in following circumstances:
- fⁿ contain loop
- static variable
- recursive
- switch state
- goto state.

```
#include <iostream>
using namespace std;

inline int cube(int i)
{
    return i*i*i;
}

int main()
{
    cout << "The cube of 3 is : " << cube(3) << endl;
    return 0;
}
```

Default Argument. (tut 25)

- It is a value provided in fn declarⁿ that is automatically assigned by compiler if the calling fn doesn't provide a value for argu.

```
#include <iostream>
using namespace std;

int sum(int x, int y, int z = 0)
{
    return (x+y+z);
}

int main()
{
    cout << sum(10, 15) << endl;
    cout << sum(10, 15, 20) << endl;
    cout << sum(10, 15, 20, 30) << endl;
    return 0;
}
```

Constant Argument

- A constant argu is one whose modification cannot take place by fn.
 - Const - keyword
- eg. `int sum(const int a, const int b)`

Proo

- Const
- compute
- main
- fn
- Data

Obj

- Word
- A
- Tre
- Dec
- error

B

- clas
- objec
- Data
- qm
- Inh
- Poly
- Dy

#

- Bet
- Pri
- M
- S

Procedure Programming.

- Consists of writing a set of instructions for the computer to follow.
- Main focus is on functions & not on flow of data.
- fn can either use local or global data.
- Data moves openly from fn to fn.

Object Oriented Programming.

- Works on concept of classes and objects.
- A class is template to create objects.
- Treats data as a critical moment.
- Decomposes problem in obj. & builds data & fn around the obj.

Basic Concepts in OOP. (tut 26)

- **classes** : Basic template for creating objects.
- **objects** : Basic run time entities.
- **Data abstraction & Encapsulation** : Wrapping data & fn into single unit.
- **Inheritance** : Prop. of one can inherited into other.
- **Polymorphism** : Ability to take more than one forms of same type.
- **Dynamic Binding** : Code which will executes is not known until the program runs.
- **Message Passing** : Obj - msg (Info) call format.

Benefits.

- Better code reusability using obj. & inheritance.
- Principle of data hiding helps build secure systems.
- Multiple objs can co-exist without any interference.
- Soft. complexity can be easily managed.

C++ Classes.

- classes in C++ is building block that leads to oop.
- It is user-defined data type, which holds its own data members & member fn.
- eg. class car

C++ Objects

- An object is an instance of class.
- When a class is defined, no memory is allocated but when it is instantiated memory is allocated.

class Syntax

```
class ClassName  
{  
    Access Specifier; // private, public, protected  
    Data members; // Variables to be used  
    Member fn {} // Method to access data member  
};
```

C++ Access Modifiers.

- Access modifiers are used to implement an imp. aspect of OOP known as Data hiding.
- There are three type of access modification.
 1. Public.
 2. Private.
 3. Protected.

1. Public.

- All the class members declared under the public specifier will be available to everyone.
- The data members & the member fn declared as public can be accessed by other classes & fn too.

2. Private.

- The class members declared as private can be accessed only by member fn inside the class.
- They are not allowed to be accessed directly by any obj. or fn outside the class.
- Only a member fn & friend fn are allowed to access private data members of class.

3. Protected.

- The protected access modifier is similar to private in the sense that it can't be accessed outside of its class unless with help of friend class.
- The difference is that the class members declared as protected can be accessed by any subclass of that class as well.

```
#include <iostream>
```

(Revision)

~~(~~1~~ 2)~~

```
using namespace std;
```

```
class employee
```

```
{
```

```
private:
```

```
int a, b, c;
```

```
public:
```

```
int d, e;
```

```
void setData (int a1, int b1, int c1);
```

```
void getData ();
```

```
cout << "a:" << a << endl;
```

```
cout << "b:" << b << endl;
```

```
cout << "c:" << c << endl;
```

```
cout << "d:" << d << endl;
```

```
cout << "e:" << e << endl;
```

```
}
```

```
};
```

```
void Employee :: setData ( int a1, int b1, int c1)
```

```
{  
    a = a1;  
    b = b1;  
    c = c1;  
}
```

```
int main()
```

```
{  
    Employee harry;  
    harry.d = 34;  
    harry.e = 89;  
    harry.setData (1, 2, 4);  
    harry.getData();  
    return 0;  
}
```

Nesting of member fn: (Revision)

```
#include <iostream> #include <string> (hd 27)  
using namespace std;
```

```
class binary
```

```
{  
    string s;
```

```
public:
```

```
    void read (void);
```

```
    void chkBin (void);
```

```
    void one-complement (void);
```

```
    void display (void);  
};
```

```
void binary :: read (void)
```

```
{  
    cout << "Enter a binary number" << endl;  
    cin << s;  
}
```

- A member fn can be called by using its name inside another member fn of some class.

void binary :: chk_bin(void)

```
{ for (int i=0; i<s.length(); i++)
```

```
{ if (s.at(i) != '0' && s.at(i) != '1')
```

```
{ cout << "Incorrect binary formed" << endl;  
  exit (0);  
}
```

```
}
```

```
}
```

```
}
```

void binary :: ones_complement(void)

```
{ for (int i=0; i<s.length(); i++)
```

```
{ if (s.at(i) == '0')
```

```
{ s.at(i) = '1';  
}
```

```
}
```

```
else
```

```
{ s.at(i) = '0';  
}
```

```
}
```

```
}
```

```
}
```

```
}
```

void binary :: display(void)

```
{ cout << "Displaying your binary no." << endl;
```

```
for (int i=0; i<s.length(); i++)
```

```
{ cout << s.at(i);  
}
```

```
{ cout << endl;  
}
```

```
}
```

int main()

```
{ binary b;
```

```
b.read();
```

```
b.chk_bin();
```

```
b.display();
```

```
b.ones_complement();
```

```
b.display();
```

```
return 0;
```

object memory allocation (Revision)

- The way memory is allocated to variables & fⁿ of class is diffⁿ even though they both are from same class.
- The memory is only allocated to variables of the class when the obj. is created.

Static Data Members & Methods in C++ (Revision) (tut 29)

- When a static data member is created, there is only a single copy of data member which is shared betⁿ all objects of class.
- It not defined by any obj. of class
- It defined outside any fⁿ using scope resolⁿ operator.
- It can only access static data members & static methods.

```
#include <iostream>
using namespace std;
```

```
class Employee
{
public:
    static int count;
    string EName;
    void setName(string name)
    {
        EName = name;
        count++;
    }
    static int getCount()
    {
        return count;
    }
};

int Employee::count = 0;

int main()
{
    Employee Harry;
    Harry.setName("Harry");
    cout << Employee::getCount() << endl;
}
```

friend functions in C++ (Revision) (tut 29)

- friend fⁿ are those fⁿ that have the right to access private data of members of class even though they are not defined inside the class.
- It is necessary to write the prototype of the friend fⁿ.

- Declaring a friend fn inside a class does make that fn a member of class

* Properties:

- Not in scope of class, means it is not a member of class

- Can be declared anywhere inside the class (be it private, public).

- It cannot access the members directly by their names, it needs (obj-name.member-name) to access any member

• Syntax:

```
class class_name
```

```
{ friend return-type fn_name (arg);
```

```
}
```

```
return-type class_name :: fn_name (arg)
```

```
{
```

```
}
```

⊕ friend classes in C++.

- friend classes are those classes that have permission to access private members of the class in which they are declared.

• Syntax:

```
class class_name
```

```
{ friend class friend_class_name;
```

```
}
```

① Constructor in C++ (tut 30)

- Constructor is a special member fn with same as of the class name.
- It is used to initialize the obj. of its class.
- It is automatically invoked.
- Constructor do not return value, hence they do not have a return type.
- It can have default arguments.
- We cannot refer to their address.

② Default Constructor. ~~(tut 30)~~

- A constructor which doesn't take any argu.
- Zero arg. constructor.

```
#include <iostream>
using namespace std;
```

```
class Construct
{
    int a, b;
public:
    Construct()
    {
        a = 10;
        b = 20;
    }
};
```

```
int main()
{
    Construct c;
    cout << "a: " << c.a << endl;
    return 1;
}
```


② Parametrized Constructor:

- It is possible to pass argu. to constructors.
- These arg. help initialize an obj. when it is created.

```
#include <iostream>
using namespace std;
class Complex
{
    int a, b;
public:
    Complex (int, int);
    void printNumber()
    {
        cout << "Your no. is " << a << " + " << b
        << "i" << endl;
    }
};
```

```
Complex :: Complex (int x, int y)
```

```
{
    a = x;
    b = y;
}
```

```
int main()
```

```
{
```

```
// implicit call
```

```
Complex a(4, 6)
```

```
// Explicit call
```

```
Complex b = Complex(5, 7);
```

```
a.printNumber();
```

```
b.printNumber();
```

```
return 0;
```

```
}
```

③ Copy Constructor

- Copy Constructor is member fn that initializes an obj. using another obj. of the same class.
- It takes a reference to an obj. of same class as an arg.

```
#include <iostream>
using namespace std;
```

```
class point {
private:
    double x, y;
public:
    point (double px, double py)
    {
        x = px;
        y = py;
    }
};
```

```
int main (void)
{
    point a [10];
    point b = point (5, 6);
    return 0;
}
```

Constructor Overloading. (tut 31)

- In C++, we can have more than one constructor in class with same name as long as each has diff. list of arg.
- A constructor is called depending upon the no. & type of arg. passed.

that
of the
same class

```
#include <iostream>
using namespace std;
```

```
class Complex
```

```
{
    int a, b;
```

```
public:
```

```
    Complex()
```

```
    {
        a = 0;
```

```
        b = 0;
```

```
    Complex (int x, int y)
```

```
    {
        a = x;
```

```
        b = y;
```

```
    }
```

```
    Complex (int x)
```

```
    {
        a = x;
```

```
        b = 0;
```

```
    }
```

```
};
```

```
int main()
```

```
{
    Complex c1 (4, 6);
```

```
    c1.printNumber();
```

```
    Complex c2 (5);
```

```
    c2.printNumber();
```

```
    Complex c3;
```

```
    c3.printNumber();
```

```
    return 0;
```

```
}
```

e com -
ng as

the

Dynamic Initialization of objects using Constructors

- Dynamic initialization of objects is done at run time. It is provided by the compiler. It is used to initialize the objects of a class. It is used to initialize the objects of a class. It is used to initialize the objects of a class.

Constructor with Default Arguments. (14/3)

```
#include <iostream>
using namespace std;

int main()
{
    class Simple
    {
        int data1;
        int data2;
    public:
        Simple (int a, int b=9)
        {
            data1 = a;
            data2 = b;
        }

        void printData();
    };

    void Simple::printData()
    {
        cout << "The value of data1 & data2 is"
        << data1 << " and " << data2 << endl;
    }

    int main()
    {
        Simple s(1);
        s.printData();
        return 0;
    }
}
```

Dynamic initialization of obj. using constructors (Rev)

- Dynamic initialization of obj. refers to initializing the obj. at run time i.e., the initial value of an obj. is provided during run time.
- It can be achieved by using constructors & by passing parameters to constructor.
- The constructor used for allocating memory at runtime is known as dynamic constructor.

Dynamic Allocation (new)

```
#include <iostream>
using namespace std;
```

```
class geeks {
    int * ptr;
public:
    geeks()
    {
        ptr = new int;
        *ptr = 10;
    }
};
```

(new)

```
void display()
{
    cout << *ptr << endl;
}
```

```
int main()
{
    geeks obj1;
    obj1.display();
    return 0;
}
```

Dynamic Deallocation (Delete)

(Delete)

(Lud 34)

```
#include <iostream>
using namespace std;
```

```
class geeks {
    int * ptr;
```

```
public:
```

```
geeks()
{
    ptr = new int;
    *ptr = 10;
}
```

```
void display()
{
    cout << "value:" << *ptr << endl;
}
```

```
int main()
{
    geeks * obj1 = new geeks();
    geeks * obj2 = new geeks();
    obj2 = obj1;
    obj1->display();
    obj2->display();
    delete obj1;
    return 0;
}
```

Destructor in C++

- A destructor is also a special member fn like a constructor.
- Destructor destroys the class obj. created by constructor.
- (~) tilde symbol.
- It is not possible to define more than one destructor.

```
#include <iostream>
```

```
using namespace std;
```

```
class Test
```

```
{
```

```
public:
```

```
Test()
```

```
{
```

```
cout << "constructor executed" << endl;
```

```
}
```

```
~Test()
```

```
{
```

```
cout << "destructor executed" << endl;
```

```
}
```

```
int main()
```

```
{
```

```
Test t;
```

```
return 0;
```

```
}
```

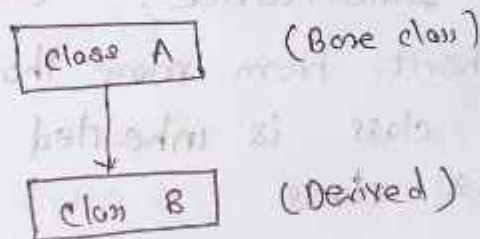

Inheritance in C++

- The capability of a class to derive properties and characteristics from another class.
- Inheritance means a new class is derived from existing class.
- Derived class - child class - New class.
- Base class - Parent class - old class.
- Derived class : The class that inherits properties from another class.
- Base class : The class whose properties are inherited.

Types of Inheritance

1. Single Inheritance (4wt 3c)

- A class is inherit from only one class
- One derived class inherited by one base class



• Syntax :

```
class A
{
};

class B : public A
{
};
```



```
#include <iostream>
using namespace std;
```

```
class Vehicle
```

```
{
public:
    Vehicle ()
```

```
{
    cout << "This is a vehicle" << endl;
```

```
}
```

```
};

class Car : public Vehicle
```

```
{
};
```

```
int main ()
```

```
{
```

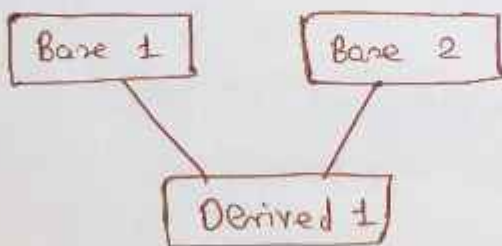
```
    Car obj;
```

```
    return 0;
```

```
}
```

2. Multiple Inheritance. (tut 37)

- A class inherit from more than one class.
- One derived class is inherited from more than one base class.



- Syntax :

```
class A
```

```
{
```

```
};
```

```
class B
```

```
{
```

```
};
```

```
class A : public B , public C
```

```
{
```

```
};
```



```

#include <iostream>
using namespace std;

class Vehicle
{
public:
    Vehicle()
    {
        cout << " This is vehicle " << endl;
    }
};

class fourwheeler
{
public:
    fourwheeler()
    {
        cout << " This is 4wheeler " << endl;
    }
};

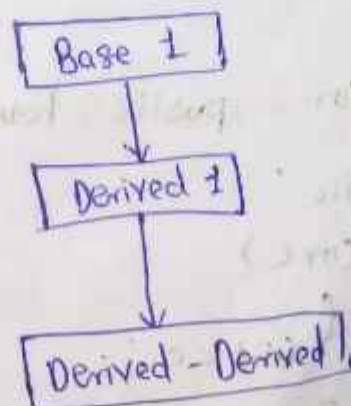
class Car : public Vehicle, public fourwheeler
{
public:
    Car()
    {
        cout << " This is Car " << endl;
    }
};

int main()
{
    Car obj;
    return 0;
}

```

3. Multilevel Inheritance (tut 38)

- A derived class is created from another derived class.



• Syntax :

```
class A
{
};
class B : public A
{
};
class C : public B
{
};
```

```
#include <iostream>
using namespace std;
```

```
class Vehicle
```

```
{
public:
    Vehicle()
    {
        cout << " This is vehicle " << endl;
    }
};
```

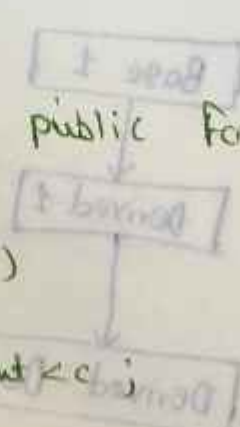
```
class 4wheeler : public Vehicle
```

```
{
public:
    4wheeler()
    {
        cout << " 4wheeler " << endl;
    }
};
```

```
class Car : public Fourwheeler
```

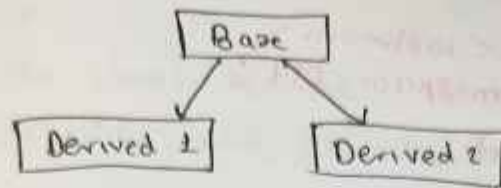
```
{
public:
    Car()
    {
        cout << "Car" << endl;
    }
};
```

```
int main()
{
    Car obj;
    return 0;
}
```



4. Hierarchical Inheritance (part 39)

- more than one derived class inherited from one base class



• Syntax

```
class A
{
}
class B : public A
{
}
class C : public A
```

```
#include <iostream>
using namespace std;
```

```
class Vehicle
{
public:
    Vehicle()
    {
        cout << "Vehicle" << endl;
    }
};

class Car : public Vehicle
{
};

class Bus : public Vehicle
{
};

int main()
{
    Car obj1;
    Bus obj2;
    return 0;
}
```

5. Hybrid Inheritance (tut 40)

- It is implemented by combining more than one type of inheritance.

```
#include <iostream>
using namespace std;

class Vehicle
{
public:
    Vehicle()
    {
        cout << "Vehicle" << endl;
    }
};

class Fone
{
public:
    Fone()
    {
        cout << "Fone" << endl;
    }
};

class Car : public Vehicle
{
};

class Bus : public Vehicle, public Fone
{
};

int main()
{
    Bus obj2;
    Car obj1;
    return 0;
}
```


Access Modifiers in C++

- Access Modifiers are used to implement an imp. aspect of OOP known as Data hiding.

1. Public :

- All the class members declared under the public specifier will be available to everyone.

2. Private :

- The class members declared as private can be accessed only by member fn inside the class.

3. Protected :

- The protected access modifier is similar to private.

- It can't be accessed outside of its class.

- It can be accessed by any derived class or that as well.

Inheritance Ambiguity in C++ (Tut 4)

- In multiple inheritance, when one class is derived from two or more base classes then there may be possibility that the base classes have functions with the same name, & the derived class may not have fn with same that name as those of its base classes.

- Ambiguity :

It can result if base & derived classes have members with the same names.

```
#include <iostream>
```

```
using namespace std;
```

```
class Base 1 {
```

```
public:
```

```
void greet()
```

```
{  
    cout << "How are you?" << endl;
```

```
}
```

```
}
```

```
class Base 2 {
```

```
public:
```

```
void greet()
```

```
{
```

```
    cout << "kaise ho?" << endl;
```

```
}
```

```
}
```

```
class Derived : public Base 1, public Base 2 {
```

```
{
```

```
int a;
```

```
public:
```

```
void greet()
```

```
{
```

```
    Base 1::greet();
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
    Base 1 base1 obj;
```

```
    Base 2 base2 obj;
```

```
    base1 obj.greet();
```

```
    base2 obj.greet();
```

```
    Derived d;
```

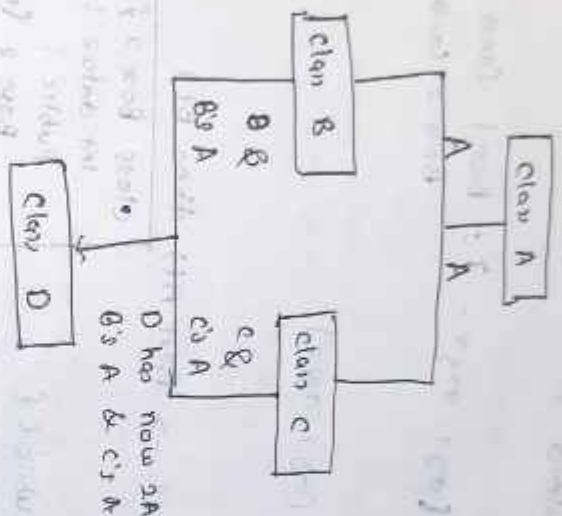
```
    d.greet();
```

```
    return 0;
```

```
}
```


Virtual Base Class in C++

Virtual Base classes are used in virtual inheritance as a way of preventing multiple "instances" of a given class appearing in an inheritance hierarchy when using multiple inheritance.



include <iostream>
using namespace std;

~~int main()~~
class A

{
public:

int a;

A();

{

a = 10;

}

}

}

}

}

class B : public Virtual A

{

public:

Virtual A

{

}

class D : public B, public A

{

}

int main()

{

D object;

cout << "a = " <<

object.a;

return 0;



Constructor in Derived Class. (part 45)

- We can use Constructors in derived classes in C++
- If base class constructor does not have any arg., there is no need of any constructor in derived class.
- But if there is one or more arg. in base class constructor, derived class need to pass arg. to base class constructor.
- In multiple & multilevel inheritance, the constructors are executed in order of inheritance.

Syntax:

Derived Constructor {arg1, arg2, ..., argN}; Base1 - Const. (arg1, arg2), Base2 - Const. (arg3, arg4)

```
{
    ...
}
Base1 - Constructor (arg1, arg2);
```

Case 1:

```
class B : public A {
    // order of execution -> First A() then B()
```

};

Case 2:

```
class A : public B, public C {
    // B(), C(), A()
```

};

Case 3:

```
class A : public B, virtual public C {
    // C(), B(), A()
```

};

#include <iostream>
using namespace std;

```
class Base1 {
    int data1;
```

```
public:
    Base1 (int i) {
```

```
        data1 = i;
```

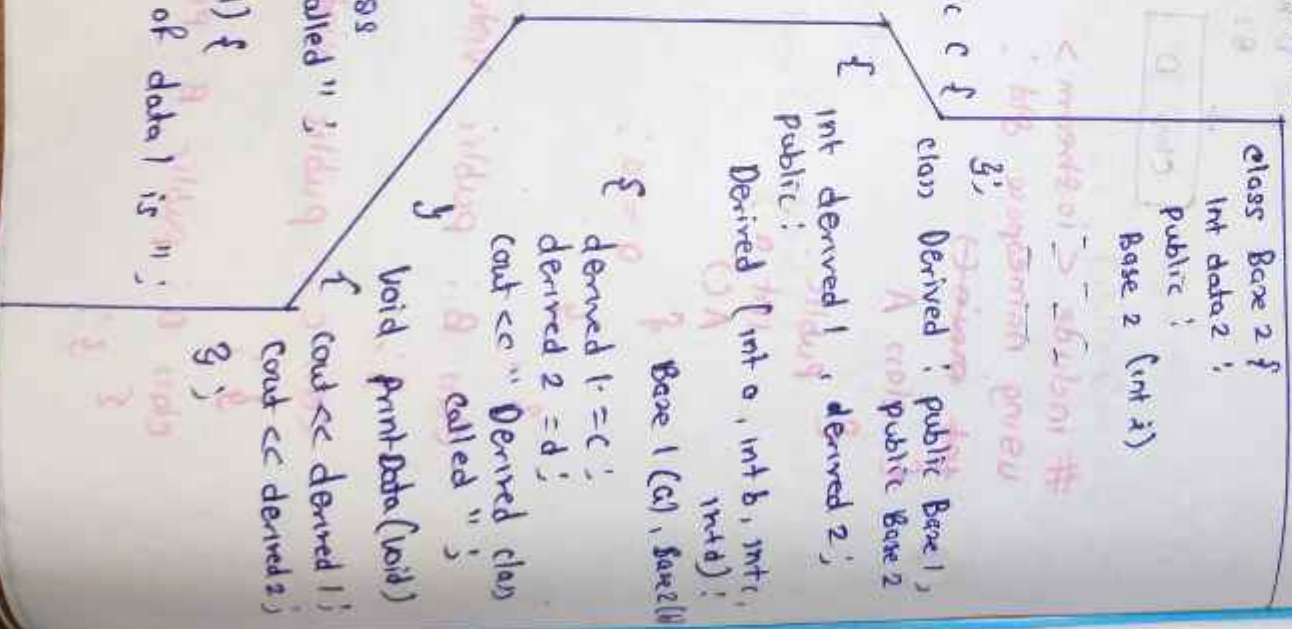
```
        cout << "Base1 class
```

```
        constructor called ";
```

```
    }
    void printData (void) {
```

```
        cout << "The value of data1 is ";
```

```
    }
};
```



445)
in C++
args, there
are constructor
are executed

(arg1, arg2)
(arg3, arg4)

```
int main()
{
    Derived hony(1,2,3,4);
    hony.printDataBase1();
    return 0;
}
```

Initializatiⁿ list in Constructors in C++

Syntax:
constructor (arg-list) : Initializatiⁿ - section

```
{ //data body
}
```

include <iostream>
using namespace std;

class Test

```
{ int a;  
  int b;
```

public :

```
Test (int i, int j) : a(i), b(j)
```

```
{ // _____ : a(i), b(i+j)
```

```
// Test (int i, int j) : b(j), a(i+b)
```

```
{ cout << "Constructor executed" << endl;
```

```
  cout << "Value of a is " << a << endl;
```

```
  _____ b = _____ + 1
```

```
} // _____
```

```
int main ()
```

```
{ Test t (4,6);
```

```
  return 0;
```

```
}
```

Revisiting Pointer ! new & delete keyword in C++

include <iostream>
using namespace std;

```
int main ()
```

```
{ int a = 4;
```

```
  int * ptr = &a;
```

```

cout << "The value of a is " << *ptr << endl;

// new keyword
int *p = new int (40);
cout << "The value at address p is " << *p << endl;

int *arr = new int [3];
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;

// Delete keyword
delete[] arr;

cout << arr[0] << endl;
cout << arr[1] << endl;
cout << arr[2] << endl;

return 0;

```



Pointers to objects & Arrow Operator in C++

#include <iostream>
using namespace std;

class Complex {

int real, imag;

public:

void getData();

cout << "The real part is " << real << endl;

cout << imag << endl;

}

void setData(int a, int b) {

real = a;

imag = b;

}

int main() {

Complex c1;

Complex *ptr = &c1; // Complex *ptr =

(*ptr).setData(1, 5); //ptr -> setData(1, 5);

(*ptr).getData(); // ptr -> getData();

}



- Polymorphism
- Inheritance
- Encapsulation

Array of Objects using Pointers (Revision)

this Pointer in C++
 It can be used to pass current obj. as parameter to another method.
 It can be used to refer current class instance variable.
 It can be used to declare indexes.

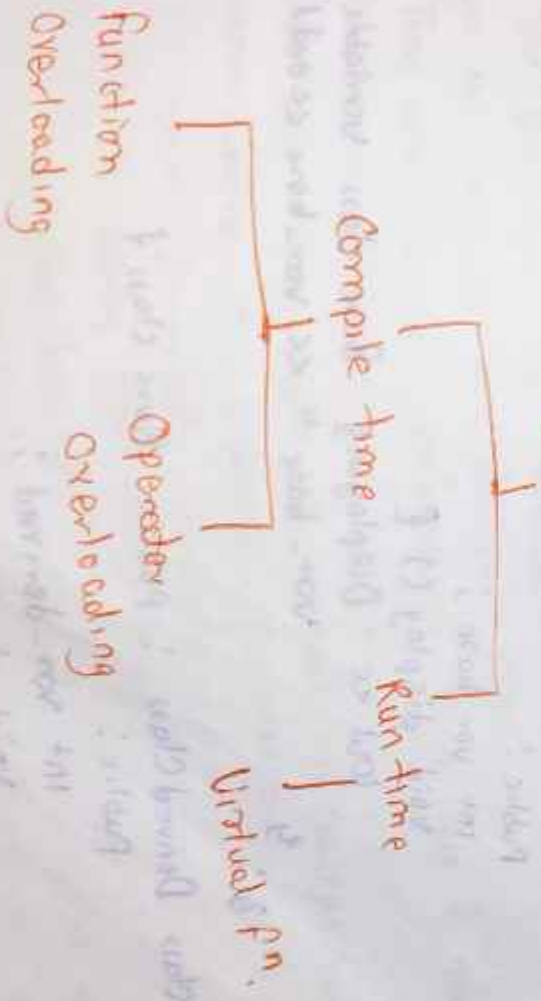
```
#include <iostream>
using namespace std;

class A {
    int a;
public:
    void addData (int a)
    {
        this->a = a;
    }
    void getData ()
    {
        cout << "Value of a : " << a;
    }
};

int main() {
    A a;
    a.addData(40);
    a.getData();
    return 0;
}
```

Polymorphism in C++ means many forms.
 - In simple words, we can define poly. as the ability of msg to be displayed in more than one form.
 - eg. person is father, husband, employee, etc.

Polymorphism



- # Compile-time
- The fn to be invoked at compile time
- overloading, early binding, static binding
- Fast execution & less flexible

Run time

- The fn to be invoked at run time
- overloading, dynamic binding, late binding
- Slow execution & more flexible

Function Overloading

(tut 4.5)

- When there are multiple fn with the same name but diffn parameters, then the fn are said to be overloaded.
- fn can be overloaded by changing no. of args. & changing the type of arg.

Operator Overloading

(tut 4.9)

- C++ has the ability to provide the operators with a special meaning for a data type.

fn Overriding

(tut 5.0)

- It occurs when a derived class has a defn for one of the member fn of base class.

Pointers to Derived class in C++

(51)

include <iostream>
using namespace std;

class BaseClass {

public:

int var_base;

void display () {

cout << "Displaying Base class variable

var_base" << endl;

};

class DerivedClass : public BaseClass {

public:

int var_derived;

void display () {


```

cout << " Displaying Base class Variable\n";
cout << " Displaying Derived << var-base var-base\n";
// << var-derived var-derived\n";
}

int main() {
    BaseClass * base-class-pointer;
    BaseClass obj-base;
    DerivedClass obj-derived;
    base-class-pointer = &obj-derived;
    base-class-pointer = &obj-derived;
    base-class-pointer -> var-base = 34;
    base-class-pointer -> display();
    return 0;
}

```

Virtual fn in C++

(not 5.2)

- It is a member fn that is declared within a base class & is re-defined by derived class.
- It is mainly used to achieve runtime poly.
- Virtual keyword.

Rules:

- Virtual fn cannot be static.
- It can be a friend fn of another class.
- It should be accessed using a ptr or reference of base class type to achieve runtime poly.
- The prototype of virtual fn should be same in base as well as the derived class & overridden.
- They are always defined in base class & overridden in derived class.
- A class may have a virtual destructor but it cannot have a virtual constructor.

```

#include <iostream>
using namespace std;

class Base {
public:
    virtual void print()

```



```

{ cout << "print base class" << endl;
  void show();
}
class derived : public base {
public:
  void print();
  { cout << "print derived class";
    void show();
    { cout << "show derived class";
  }
}

```

```

int main() {
  base * bptr;
  derived d;
  bptr = &d;
  bptr -> print();
  bptr -> show();
  return 0;
}

```

Pure Virtual FN & Abstract classes in C++

- Sometimes implementⁿ of all FN cannot be provided in base class because we don't know the implemⁿ. Such a class is called as **abstract class**.
- A Pure Virtual FN in C++ is virtual FN for which we can have an implementⁿ, but we must override that FN in derived class, otherwise the derived class will also become an abstract class.
- A pure virtual FN is declared by assigning 0 in the declaration.


```

syntax:
class class_name // abstract class
{
public:
// pure virtual fn
}

```

#include <iostream>
using namespace std;

```

class Base {
public:
    int x;
    virtual void fun() = 0;
}

```

```

{
    get x();
    return x;
}

class Derived : public Base {
public:
    int y;
    void fun()
    {
        cout << "fun() called";
    }
}

```

```

{
    int main(void)
    {
        Derived d;
        d.fun();
        return 0;
    }
}

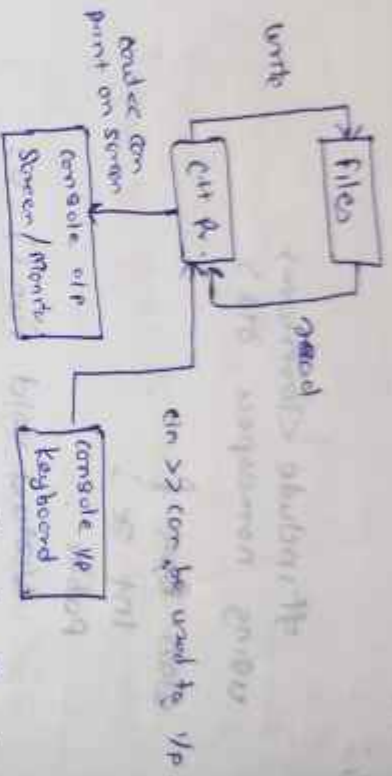
```

ASCII Table (American Std code for Info Interchange)

- It is a common character encoding format for text data in computers & on the Internet (established in 1963)
- A-Z = 65 to 90
- 0-9 = 48 to 57

File Handling and Files in C++

- It is used to store data permanently in computer.
- Using file handling we can store our data in secondary memory (hard disk).
- Streams



- The sequence of bytes given as i/p to the executing program & the seq. of bytes that comes as o/p from the executing program is called as 'stream'.
- In other words, streams are nothing but the flow of data in sequence.
- In i/p & o/p operatⁿ betⁿ the executing program & the devices like keyboard & monitor are known as 'console i/o operation'.
- The i/p & o/p operatⁿ betⁿ the executing program & file are known as 'disk i/o operation'.

① ios

- It stand for i/o stream.
- This class is the base class for other classes in this class hierarchy.
- This class contains the necessary facilities that are used by all the other derived classes for i/p & o/p operatⁿ.

② istream (input stream)

- This is derived from ~~the~~ class 'ios'.
- This handle i/p stream.
- Extraction operators (>>) is overloaded.
- This class declares i/p fn such as get(), getln(), & read().

in computer
in section

to VP.

executing
c/p from

the flow of

program &
run as

program &

in this

one
& c/p

getting

o ostream (output stream)

o This is derived from class 'ios'.

o This handles output stream.

o Insertion operator (<<)

o This class declares o/p fn such as put() & write()

o ostreambuf

o This class contains a ptr which points to buffer which is used to manage the VP & o/p streams

o ostreambase

o This class provides open() operation

o Serves as base for fstream, ifstream & ofstream class.

o This class contains open() & close() fn

o ifstream

o This class provides VP operations

o It contains open() fn with default VP mode

o Inherits the fn get(), getline(), read(), seekg() & tellg() fn from istream.

o ofstream

o This class provides o/p operations.

o It contains open() fn with default o/p mode

o Inherits the fn put(), write(), seekp() & tellp() fn from the ostream.

o fstream

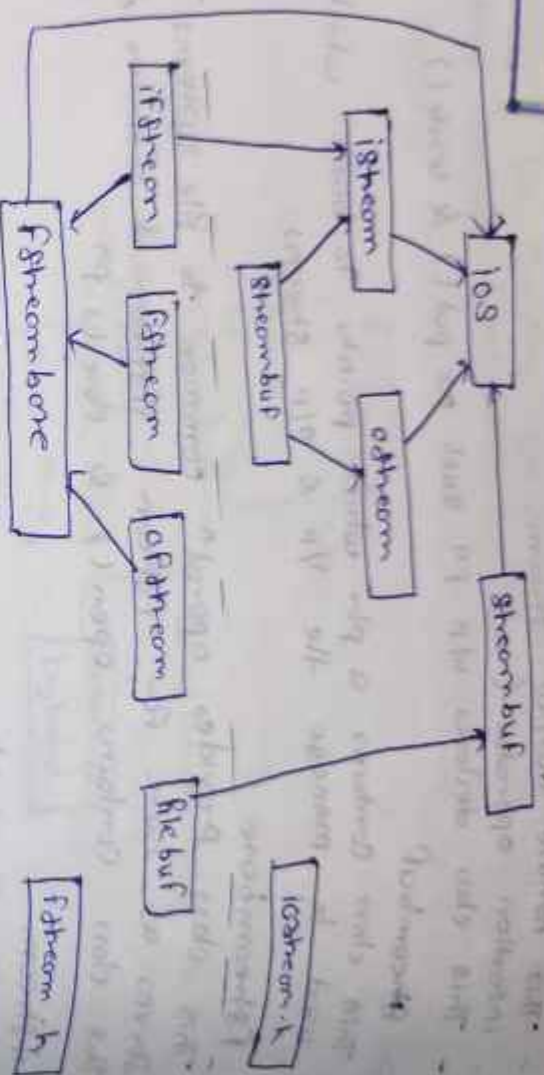
o This class provides support for simultaneous VP & o/p operation.

o Inherits all the fn from ifstream & ofstream classes through istream.

o Filebuf

o Its purpose is to set the file buffers to read & write.

o We can also use File buffer member fn to determine the length of file



Write and Read File using Constructor (Part 5)

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main() {
    string s1 = "Hong Bhai";
    ofstream out("Sample60.txt");
    out << s1;
```

```
    ifstream in("Sample60b.txt");
    getline(in, s2);
    cout << s2;
    return 0;
```

3

Write & Read File in Same File (Part 5)

```
#include <iostream>
#include <fstream>
using namespace std;
```



```

int main() {
    ofstream head ("Samplea.txt");
    string name;
    cout << "Enter your name:";
    cin >> name;
    head << name + " is my name";
    head.close();
    ifstream hin ("Sample.txt");
    string content;
    hin >> content;
    cout << "The content of this file is : " << content;
    hin.close();
    return 0;
}

```

Open() & eof() fn. (4th 5th)

```

# include <iostream>
# include <fstream>
# include <string>
using namespace std;
int main() {

```

```

    ofstream out;
    out.open ("Sampleb.txt");
    out << "This is me ";
    out.close();
    ifstream in;
    string str;
    in.open ("Sampleb.txt");
    while (in.eof() == 0) {

```

```

        getline(in, str);
        cout << str << endl;
    }
    in.close();
    return 0;
}

```

Templates in C++ (1st-57) (1st-57)

- A template is simple idea is to pass the data types as a parameter so that we don't need to write the same code for diffⁿ data types.
- keyword: 'template' & 'type name'.
- The second keyword can always be replaced by keyword 'class'.

Function Template

- Generic fn use the concept of fn template.
- Generic fn define a 'set of operatⁿ' that can be applied to the various types of data.

Syntax:

```
template < class T type > ret-type fun-name (parameters)
{
}
```

include <iostream>
using namespace std;

template < class T > T add (T &a, T &b)
{
 T result = a + b;
 return result;
}

int main()
{

int i = 2;
int j = 8;

float m = 2.3;
float n = 1.2;

cout << " Addⁿ of i & j is " << add(i, j);
cout << " Addⁿ of m & n is " << add(m, n);
return 0;

}

Class Template

- It is similar to the fn template
- When a class uses the concept of Template then the class is known as generic class

Syntax

```
template <class T type>  
class class-name
```

```
{  
}
```

Ttype is a placeholder name which will be determined when the class is instantiated.

```
#include <iostream>  
using namespace std;
```

```
template <class T>
```

```
class A
```

```
{  
public :
```

```
    T num 1 = 5;
```

```
    T num 2 = 6 ;
```

```
    void add ()
```

```
{  
    cout << num 1 + num 2 ;
```

```
};
```

```
int main ()
```

```
{  
    A <int> d;
```

```
    d.add ();
```

```
    return 0;
```

```
}
```

class Template with Multiple Parameters

Syntax:

```
template < class T1, class T2, ... >
```

```
class class_name
```

```
{  
  ...  
}
```

```
#include <iostream>
```

```
using namespace std;
```

```
template < class T1, class T2 >
```

```
class A
```

```
{
```

```
    T1 a;
```

```
    T2 b;
```

```
public:
```

```
    A (T1 x, T2 y)
```

```
{
```

```
    a = x;
```

```
    b = y;
```

```
}
```

```
    void display()
```

```
{
```

```
        cout << "value : " << a << b << endl;
```

```
}
```

```
int main()
```

```
{
```

```
    A <int, float> d (5, 6.5);
```

```
    d.display();
```

```
    return 0;
```

```
}
```


Parameters

fn template with multiple parameters

Syntax:

```
template < class T1, class T2, ... >
return-type fun_name (arg of type T1, T2, ...)
{
}
```

```
#include <iostream>
```

```
using namespace std;
```

```
template < class X, class Y >
```

```
void fun (Xa, Yb)
```

```
{
```

```
    cout << "value of a" << a << endl;
```

```
    cout << b << endl;
```

```
}
```

```
int main()
```

```
{ fun (15, 12.3);
```

```
    return 0;
```

```
}
```

Default Parameters for templates

```
#include <iostream>
```

```
using namespace std;
```

```
template < class T, class U = char >
```

```
class A
```

```
{ public:
```

```
    T x;
```

```
    U y;
```

```
};
```

```
int main()
```

```
{ A <char> a;
```

```
  A <int, int> b;
```

```
  cout << size of (a);
```

```
  cout << size of (b);
```

```
  return 0;
```

```
}
```

Template fn Overloading

- The name of fn templates are same but called with diffⁿ args.
- When fn template is overloaded with non-template fn, the fn name remains same but fn args are unlike.

```
#include <iostream>
using namespace std;

template <class T>
void display (T t1)
{
    cout << "Displaying Template : " << t1;
}

void display (int t1)
{
    cout << "Explicitly display : " << t1;
}

int main ()
{
    display (200);
    display (12.40);
    display ('G');
    return 0;
}
```


★ STL in C++

- STL stands for std. library Template Library
- STL is a set of C++ template classes to provide common programming data structures & fⁿ such as lists, stack, array etc.
- It is a library of container classes, algorithms, & iterators.

* Components of STL :

① **Containers** : The STL provides a range of containers, such as vector, list, map, set & stack, which can be used to store & manipulate data.

② **Algorithm** : The STL provides a range of algorithms, such as sort, find & binary-search, which can be used to manipulate data stored in containers.

③ **Iterators** : Iterators are obj. that provides a way to traverse the elements of containers.

Iterator = forward, bidirectional & random access iterator.

④ **Functⁿ objects** : (tw6?) fⁿ obj, also known as functors, are objects that can be used as fⁿ arguments to algorithm.

They provide a way to pass a fⁿ to an algorithm, allowing you to customize its behavior.

⑤ **Adapters** : Adapters are components that modify the behavior of other components in STL.

⑥ **function** :

SILVER

Containers in C++ STL

- (defn) - (last page).
- A container is a holder object that stores a collection of other objects.

1. Sequence Containers

- Sequence Containers implement data structures that can be accessed sequentially.

- array : Static contiguous array (class template)
- Vector : Dynamic contiguous array (class template)
- deque : Double ended queue (class template)
- forward list : Singly - linked list (class template)
- list : Doubly - linked list (class template)

2. Associative Containers

- It implement sorted data structures that can be quickly searched ($O(\log n)$)

- Set : collectⁿ of unique keys, sorted by keys (class template)
- Map : Collectⁿ of key-value pairs, sorted by keys, keys are unique (class temp).

• Multiset : Collection of keys, sorted by keys. (class temp.)

• Multimap : Collection of key-value pairs, sorted by template

3. Derived Container (Container adapters)

- It provides a different interface for sequential containers.

• Stack : Adapts a container to provide stack (LIFO d.s.) (class temp.)

• Queue : Adapts a container to provide queue. (FIFO d.s.) (class temp.)

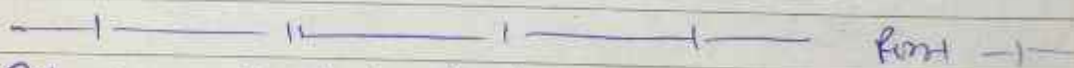
• Priority queue : Adapts a container to provide priority queue. (class template)

// Vectors in STL (that sk)

- Vectors are the same as dynamic arrays with the ability to resize itself automatically when an element is inserted or deleted, with their storage being handled automatically by container.
- `std::vector`.
- It is class template that contains the vector container & its member fn.
- It is defined inside the `<vector>` header file.
- The member fn of `std::vector` class provide various functionalities to vector containers.

• Member fn in Vectors

* Iterators

1. `begin()` : pointing to first element
2. `end()` : pointing to last element
3. `rbegin()` : Returns a reverse iterators pointing to last element.
4. `rend()` :  `first`
5. `cbegin()` : Returns a constant iterator pointing to first element
6. `rend()` : last element
7. `crbegin()` : Returns a constant reverse iterator pointing to last element
8. `crend()` :

Shadhy :

Vector In

① Size & Capacity

② push, back & pop, back

③ emplace, back

④ at, at []

⑤ front & back

⑥ erase

⑦ insert

⑧ clear

⑨ empty

no. of elements in vector

* Capacity

- (i) `size()` : Returns no. of elements in vector
- (ii) `max_size()`
- (iii) `capacity()` : Returns the size of storage space currently allocated to vector expressed as no. of elements
- (iv) `resize(n)`
- (v) `empty()`
- (vi) `reserve()` : Request that the vector capacity be at least enough to contain n elements.

* Element access :

- (i) Reference operator (`g`)
- (ii) `at(g)`
- (iii) `front()`
- (iv) `back()`
- (v) `data()`

* Modifiers

- (i) `assign()`
- (ii) `push-back()`
- (iii) `pop-back()`
- (iv) `insert()`
- (v) `erase()`
- (vi) `swap()`
- (vii) `clear()`
- (viii) `emplace()`

List in STL (List Sq)

- List one sequence container that allow non-contiguous memory allocation.
- As compared to vector, the list has slow traversal.
- It is defined as template so it is able to hold any data type.

* Std :: list

- (i) `front()`
- (ii) `back()`
- (iii) `push-front()`
- (iv) `push-back()`
- (v) `pop-front()`
- (vi) `pop-back()`
- (vii) `insert()`
- (viii) `size()`
- (ix) `begin()`
- (x) `end()`

Map in STL (Tut 60)

- Maps are associative containers that store elements in a mapped fashion.
- Each element has a key value & mapped value.
- No two mapped values can have same key values.

(i) begin() (ii) end() (iii) size() (iv) max-size() (v) empty()
(vi) erase() (vii) clear()

Deque in STL - Double-ended-queue. (Tut 60)

- In this, elements can be added & removed from both ends.

include <deque>

deque<int> d = {1, 2, 3, 4}

- push-back & push-front
- emplace-back & emplace-front
- pop-back & pop-front

size, erase, clear, begin, end, rbegin, rend, insert, front, back

Pair in STL (Tut 61)

- It is used to combine together two values that may be of different data types or same data types.

pair<T₁, T₂> p = {1, 2, 3}

pair<char, int> p = {'a', 11}

Set in STL (as per)

Stack in STL (as per)

- It is used to store multiple elements in specific order

LIFO

#include <stack>



push, pop, top

size, empty, swap

Queue in STL (as per)



- Store multiple element (FIFO) added from end, removed from back

#include <queue>

push, pop, front, back, size, empty, swap

push, pop, front, back, size, empty, swap

empty, swap

swap

Queue (as per)

- A type of queue that removed element based on their priority value

#include <priority_queue>

priority_queue <int>

Some

- It is used to store multiple elements in specific order

Stack in STL (tut 62)

Set in STL (tut 62)

- A Set is a data structure that store unique elements of some type in sorted order.

Set <int> S;

• insert, erase

• error

• find

• count

• size, empty, erase

- Types Study (self)

Algorithm :

• Sorting

• Reverse

• Permutation

• Swap, min, max

• Binary Search

• Count Set bits